The Problem: Traditional Backend Models

As we delve deeper into microservices patterns, it's important to understand the challenges posed by traditional models that led to the advent of patterns like the Backends for Frontends.

One-Size-Fits-All Approach

In traditional backend models, the backend is a monolithic entity serving multiple frontends — desktop, mobile, and other client-facing applications. It provides a uniform set of APIs for all frontends, regardless of their unique requirements.

Think of a scenario where a desktop application and a mobile application are accessing the same API from a traditional backend. The desktop app with its high processing power and larger display can handle a substantial amount of data. But what about the mobile app? It has limited processing power, a smaller screen, and possibly a restricted data plan. Should it receive the same amount of data as the desktop app? Is it efficient to process all that data in a mobile environment? Not quite.

Performance Inefficiency and Over-fetching

This leads us to another problem — performance inefficiency and over-fetching. Over-fetching happens when a client downloads more information than it actually needs. Over-fetching in traditional backend models can lead to performance issues, longer loading times, and increased bandwidth usage.

Complexity in Code Management

When a single codebase serves multiple frontends, it becomes bloated and complex. It's harder to maintain, troubleshoot, and update. The more complex the system, the more time and resources it consumes, which could be used for other productive tasks.

Lack of Flexibility and Scalability

Lastly, traditional backends lack flexibility and scalability.

With rapidly evolving user expectations and technological advancements, applications need to adapt quickly. Traditional backends, with their rigid structure and complexity, make it harder for businesses to innovate and scale.

Imagine having to sift through a monolithic codebase to change a small feature for one specific frontend. Or scaling the entire backend system just because one frontend is experiencing high traffic. It sounds like an overkill, right? That's because it is.

So, it’s evident that the traditional backend models pose a series of challenges. They're like outdated machinery that can't keep up with modern demands. We need a new approach, something that caters to the unique requirements of different frontends while ensuring efficiency and scalability. The BFF help us resolve these issues.